

Network Automation and Digital Twin Lab

Multi-Vendor Config via YANG and NETCONF

Presented by: Saeed Ali, Network Engineer

Neu-Isenburg, 18/06/2026



The Problem

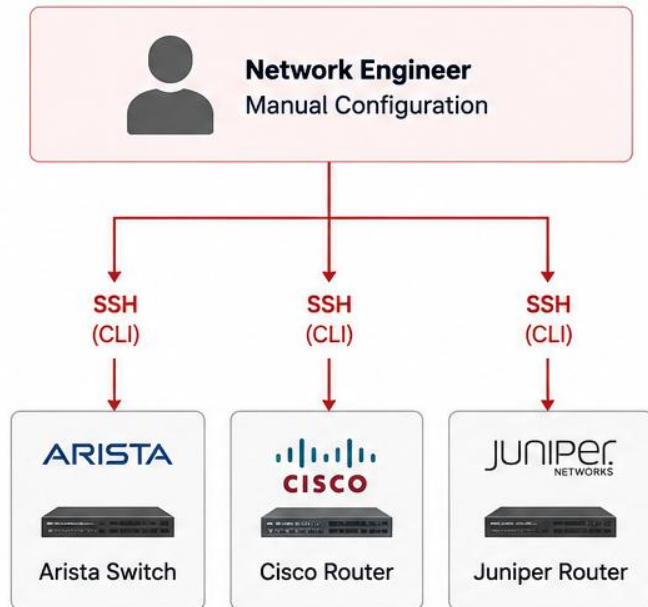
- Each vendor has different CLI syntax
- Manual configuration across devices
- No version control for network configs
- Configuration drift over time
- Slow to scale (100s of devices)

The Solution: YANG + NETCONF

- ✓ Unified data model (YANG)
- ✓ Standardized transport protocol (NETCONF)
- ✓ Works for Arista AND Cisco (any vendor)
- ✓ Configuration as Code (Git)
- ✓ Automated deployment & rollback

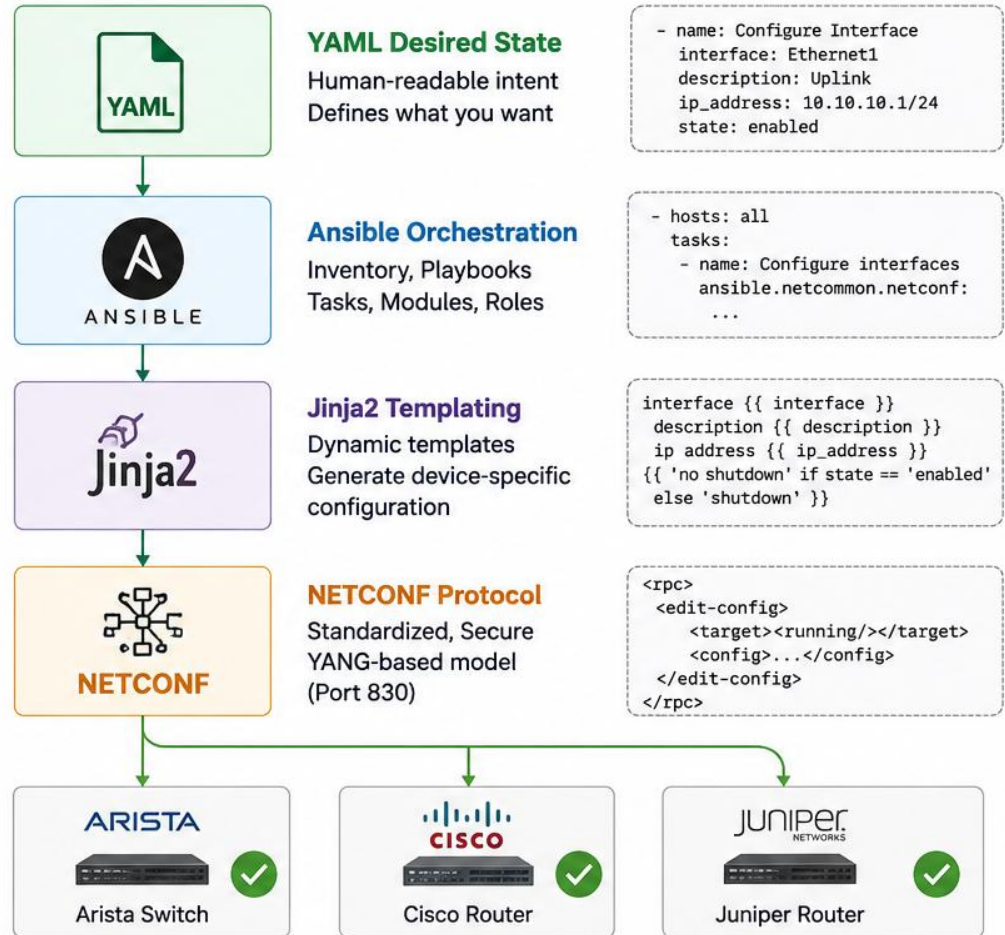
Architecture

1. TRADITIONAL MANUAL APPROACH



- Device-specific CLI knowledge required
- Manual & time-consuming
- Error-prone
- Not scalable
- Hard to maintain & audit

2. MODERN AUTOMATION ARCHITECTURE



Vendor
Neutral



Automated
& Repeatable



Consistent
& Reliable



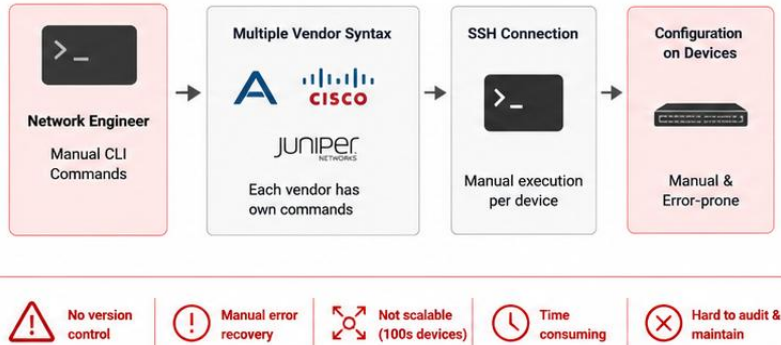
Scalable



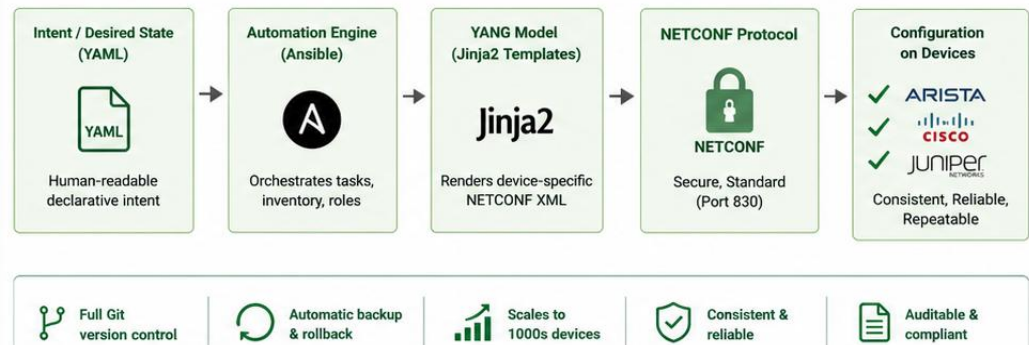
Auditable
& Versioned

YANG Model vs Traditional CLI

TRADITIONAL CLI AUTOMATION



YANG + NETCONF AUTOMATION



YANG MODEL vs TRADITIONAL MODEL

TRADITIONAL MODEL (Unstructured)

- ❌ Device-specific CLI commands
- ❌ No standard data model
- ❌ Screen scraping / Text parsing
- ❌ Hard to validate
- ❌ Vendor lock-in
- ❌ Difficult integration
- ❌ Inconsistent across devices

Example (Cisco CLI)

```
interface Ethernet1
description Uplink
ip address 10.10.10.1 255.255.255.0
no shutdown
!
```

⚠️ Unstructured, free-form text
Hard to automate & validate

YANG MODEL (Structured & Standard)

- ✅ Standardized data model (IETF)
- ✅ Structured, hierarchical data
- ✅ Machine-readable (XML/JSON)
- ✅ Easy to validate
- ✅ Vendor neutral
- ✅ Easy integration (APIs, Tools)
- ✅ Consistent across devices

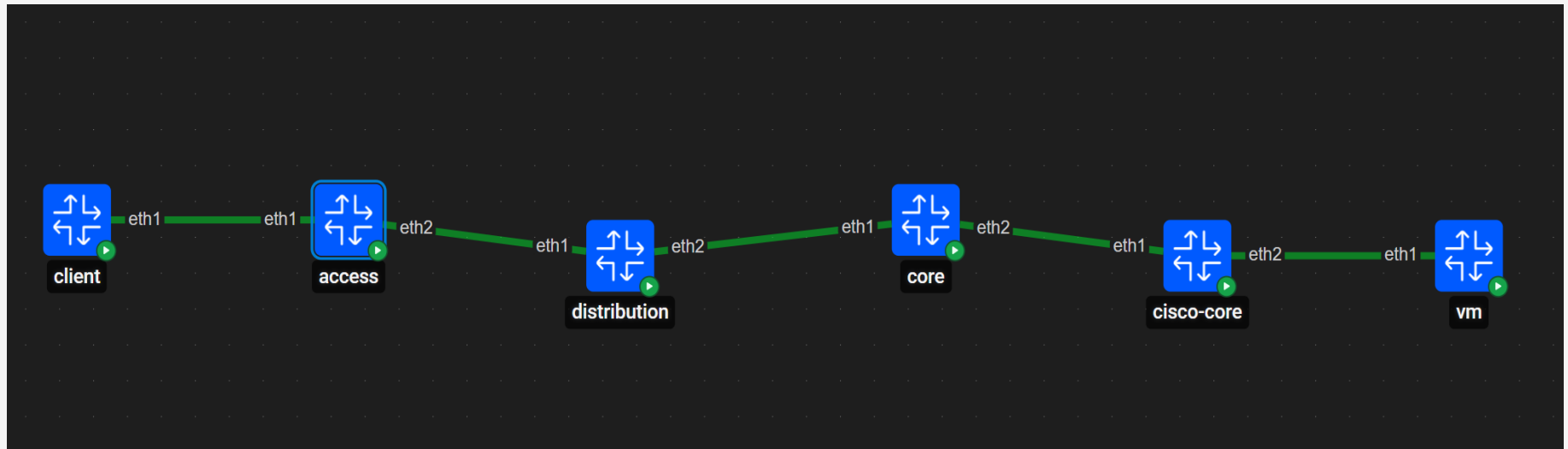
Example (YANG / NETCONF XML)

```
<interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
  <interface>
    <name>Ethernet1</name>
    <description>Uplink</description>
    <enabled>true</enabled>
    <ipv4>
      <address>
        <ip>10.10.10.1</ip>
        <netmask>255.255.255.0</netmask>
      </address>
    </ipv4>
  </interface>
</interfaces>
```



✅ Structured, consistent
Easy to automate & validate

Lab - Diagram



Demo Flow

- 1 Deploy ContainerLab (Arista + Cisco)
- 2 Show YAML config vars (human-readable)
- 3 Render YANG XML via Jinja2 (single template, multiple vendors)
- 4 Push to devices via NETCONF (live)
- 5 Verify configuration (show commands)
- 6 Demonstrate rollback

Key Features

- ◆ Multi-Vendor: Same code, Arista & Cisco
- ◆ Declarative: Define desired state (not HOW to do it)
- ◆ Atomic: NETCONF transaction (all or nothing)
- ◆ Modular: Add new nodes = add new YAML file
- ◆ Safe: Automatic backup before each push

Config as Code (YAML)

Access Switch - First Floor

```
interfaces:
  - name: Ethernet1
    description: "Link to client"
    enabled: true
    ipv4:
      - ip: "10.10.10.1"
        prefix_length: 24
  - name: Ethernet2
    description: "Link to distribution"
    enabled: true
    ipv4:
      - ip: "10.10.20.1"
        prefix_length: 24
```

Access Switch - Second Floor

```
interfaces:
:
  prefix_length: 24
```

Business Benefits



Faster deployment (minutes vs hours)



Reduced human errors (zero-touch config)



Full audit trail (Git history)



Easy to scale (add device = add YAML)



Vendor-agnostic (future-proof)

Safety: Backup & Rollback

-  Automatic pre-push backup
-  One-command rollback to previous state
-  Configuration snapshots (timestamped)
-  Verified before pushing to production
-  Audit log of all changes

Next Steps (Post-Demo)

-  Integrate with Nautobot (IPAM source of truth)
-  Add compliance checks (before push)
-  Add monitoring for config drift
-  Add approval workflow (prod gates)
-  Expand to BGP, MPLS, QoS configs

One Config
Multiple Vendors
Infrastructure as Code